

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)*

Assessment Tool Weightage: 40%

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 50

Code of Conduct

I __ G. Ram Charan (192525043) ____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: __ G. Ram Charan _____

Criteria	Problem Interpretation (2)	Analytical Reasoning (2.5)	Method Selection (2)	Solution Quality (2)	Presentation of Analysis (1.5)	TOTAL (10)
Weightage	20%	25%	20%	20%	15%	100%
Q1						
Q2						
Q3						
Q4						
Q5						

Signature of the Faculty

Total Marks Awarded:

Analytical Problem Solving

1. A cloud datacenter host has 64 CPU cores and supports CPU overcommitment at a ratio of 2.5:1. Each virtual machine (VM) requires 4 vCPUs. Calculate the maximum number of VMs that can be deployed on this host. If actual CPU utilization reaches 85%, analyze whether further VM allocation is advisable.
2. A virtualized storage system provides 20 TB of physical storage with thin provisioning at a ratio of 3:1. Each VM is allocated 500 GB of virtual disk space. Calculate the maximum number of VMs that can be provisioned.
3. Evaluate the security implications of multi-tenancy in cloud datacenters. Analyze how virtualization can both enable and weaken isolation, and recommend mechanisms to strengthen tenant separation.
4. Analyze the role of Service-Oriented Architecture (SOA) in enabling interoperability across heterogeneous cloud platforms. Identify key challenges such as service latency, security, and version control, and propose solutions.
5. A cloud system implementing presence services faces scalability issues during peak usage. Analyze the architectural bottlenecks and recommend design improvements to ensure real-time updates and efficient resource utilization.

Solution:-

1) Given:

- Physical CPU cores = 64
- CPU overcommitment ratio = 2.5 : 1
- Each VM requires = 4 vCPUs

Step 1: Calculate the total available vCPUs

Total vCPUs = Physical CPU cores $\times$ Overcommitment ratio

= 64×2.5

= **160 vCPUs**

Step 2: Calculate the maximum number of VMs

Number of VMs = Total vCPUs $\div$ vCPUs required per VM

= $160 \div 4$

= **40 VMs**

Analysis:

The CPU utilization has reached **85%**, which means most of the CPU resources are already being used. If more VMs are added, the CPU may become overloaded, causing slower performance, increased response time, and possible SLA violations.

Conclusion:

Maximum number of VMs = 40.

Since the CPU utilization is already **85%**, **it is not advisable to allocate more VMs** because it may affect the overall performance of the cloud host.

2) Given:

- Physical storage = **20 TB**
- Thin provisioning ratio = **3 : 1**
- Storage allocated per VM = **500 GB**

Step 1: Calculate the total virtual storage

Virtual Storage = Physical Storage $\times$ Thin Provisioning Ratio

= 20 TB $\times$ 3

= **60 TB**

Step 2: Convert TB into GB

1 TB = 1000 GB

60 TB = 60 $\times$ 1000

= **60,000 GB**

Step 3: Calculate the maximum number of VMs

Number of VMs = Total Virtual Storage $\div$ Storage per VM

= 60,000 GB $\div$ 500 GB

= **120 VMs**

Analysis:

Thin provisioning allows more virtual storage to be allocated than the available physical storage because storage is assigned only when it is actually used. This improves storage utilization and reduces storage wastage. However, if many VMs start using their full allocated storage at the same time, the physical storage may become full, which can affect system performance.

Conclusion:

The maximum number of VMs that can be provisioned is 120.

Thin provisioning improves storage efficiency, but the storage usage should be monitored regularly to avoid running out of physical storage.

3) Introduction:

Multi-tenancy is a cloud computing model where multiple customers (tenants) share the same physical server while using separate virtual machines (VMs). Virtualization provides isolation between tenants, but it also introduces certain security risks.

Virtualization Enables Isolation:

- Each tenant runs in a separate Virtual Machine (VM).
- Resources such as CPU, memory, and storage are logically separated.
- One tenant cannot directly access another tenant's operating system or applications.
- This improves resource utilization while maintaining basic security.

How Virtualization Can Weaken Isolation:

Even though VMs are isolated, some security issues may occur, such as:

- **Hypervisor attacks:** If the hypervisor is compromised, all VMs on the host may be affected.
- **VM Escape:** A malicious VM may escape its virtual environment and access the host system.
- **Data Leakage:** Shared resources may expose sensitive information between tenants.
- **Side-Channel Attacks:** Attackers may obtain information by observing shared hardware resources like CPU cache.

Methods to Strengthen Tenant Separation:

To improve security in a multi-tenant cloud environment, the following methods can be used:

- Use a secure and updated hypervisor.
- Encrypt data during storage and transmission.
- Implement Multi-Factor Authentication (MFA).
- Use Virtual LANs (VLANs) to isolate network traffic.
- Deploy firewalls and Intrusion Detection Systems (IDS).
- Regularly monitor and update cloud infrastructure.
- Apply strict access control policies for users.

Conclusion:

Virtualization provides efficient resource sharing and tenant isolation in cloud datacenters. However, security risks such as hypervisor attacks, VM escape, and data leakage can weaken isolation. These risks can be reduced by using strong security mechanisms like encryption, secure hypervisors, network isolation, firewalls, and continuous monitoring. Therefore, proper security practices are essential to maintain safe tenant separation in cloud environments.

4) Introduction:

Service-Oriented Architecture (SOA) is a software design approach in which applications communicate with each other through services. It helps different cloud platforms work together, even if they are developed using different programming languages or operating systems.

Role of SOA in Cloud Interoperability:

- SOA enables communication between different cloud platforms.
- It allows services to be reused in different applications.
- It supports easy integration of applications from different vendors.
- It improves scalability and flexibility by using independent services.
- It reduces dependency between applications through loose coupling.

Challenges and Solutions:

1. Service Latency

Challenge:

Communication between services over the network may increase response time.

Solution:

- Use caching techniques.
- Apply load balancing.
- Optimize network communication.

2. Security

Challenge:

Unauthorized users may access cloud services, leading to data theft or misuse.

Solution:

- Use authentication and authorization.
- Encrypt data during transmission.
- Use secure communication protocols such as HTTPS/TLS.
- Deploy API gateways and firewalls.

3. Version Control

Challenge:

Updating one service may affect applications using older versions.

Solution:

- Maintain multiple API versions.

- Ensure backward compatibility.
- Properly test new versions before deployment.

Analysis:

SOA improves interoperability by allowing different cloud services to exchange information easily. Although issues like latency, security, and version control exist, they can be managed with proper design and security practices. This makes cloud applications more reliable and efficient.

Conclusion:

SOA plays an important role in connecting heterogeneous cloud platforms by enabling seamless communication between services. Challenges such as latency, security, and version control can be overcome through caching, encryption, authentication, API versioning, and proper service management. Therefore, SOA is an effective approach for building scalable and interoperable cloud applications.

5) Introduction:

Presence services are used in cloud applications to show the real-time status of users, such as online, offline, busy, or available. During peak usage, a large number of users access the system simultaneously, which creates scalability challenges.

Architectural Bottlenecks:

The following are the major bottlenecks that affect the performance of presence services:

- A single server handling all user requests becomes overloaded.
- A large number of simultaneous user connections increases server load.
- Frequent status updates create heavy network traffic.
- Database servers receive too many read and write requests.
- High resource utilization causes increased response time and delays.

Recommended Design Improvements:

To improve scalability and ensure efficient resource utilization, the following methods can be implemented:

- Use **load balancing** to distribute user requests across multiple servers.
- Implement **horizontal scaling** by adding more servers when the number of users increases.
- Use **distributed databases** to reduce database bottlenecks.
- Store frequently accessed data in **cache memory** to reduce database access.
- Use **message queues or publish-subscribe (Pub/Sub)** architecture for real-time status updates.
- Enable **auto-scaling** so cloud resources increase automatically during peak traffic.

- Continuously monitor system performance to detect and resolve issues quickly.

Analysis:

The main reason for scalability issues is the sudden increase in user requests and real-time updates. By distributing the workload among multiple servers, reducing database access through caching, and automatically scaling cloud resources, the system can provide faster response times and better resource utilization.

Conclusion:

Presence services can efficiently handle peak usage by using load balancing, horizontal scaling, distributed databases, caching, auto-scaling, and message queue technologies. These improvements ensure real-time updates, better performance, high availability, and efficient utilization of cloud resources.